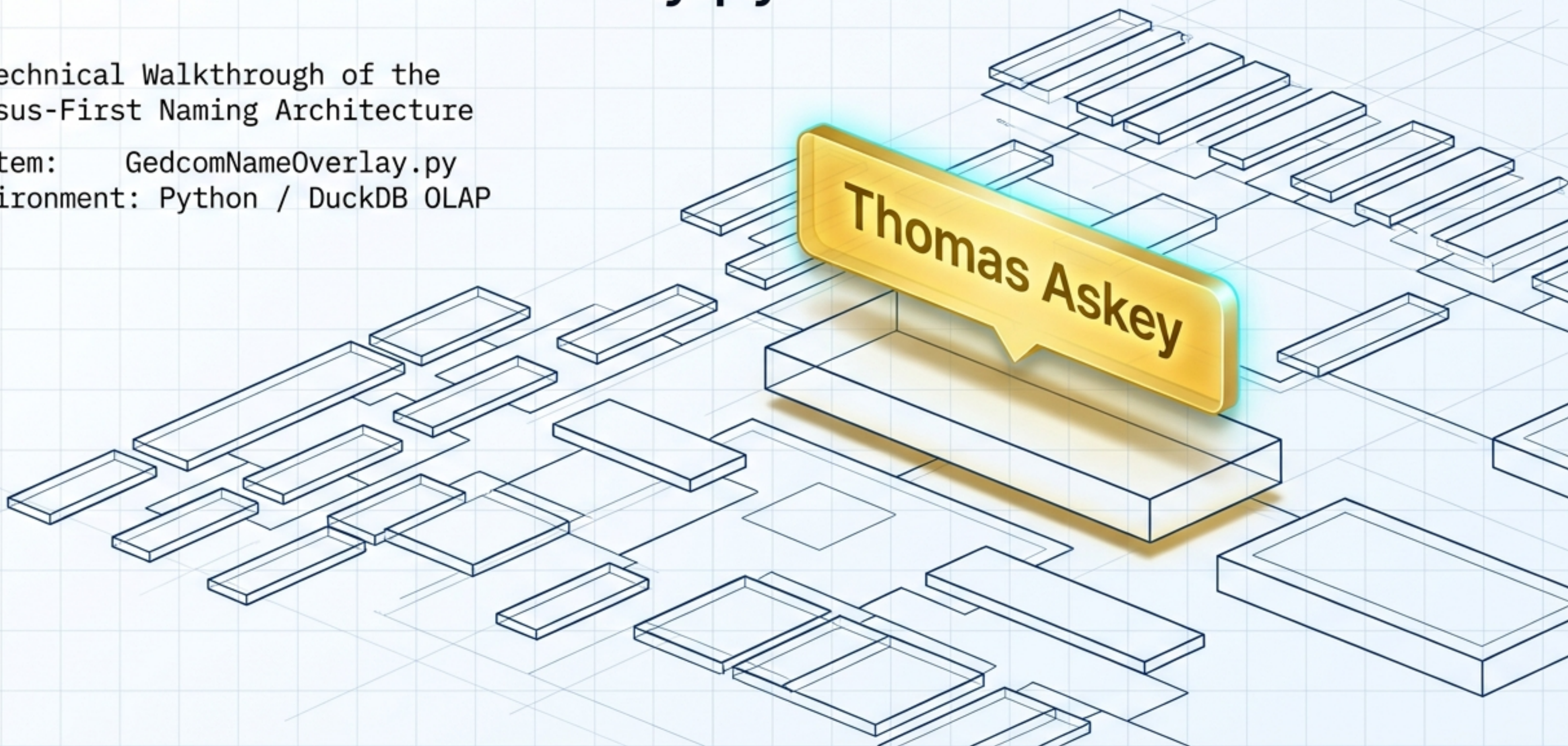


The Label-Maker: Design Documentation for GedcomNameOverlay.py

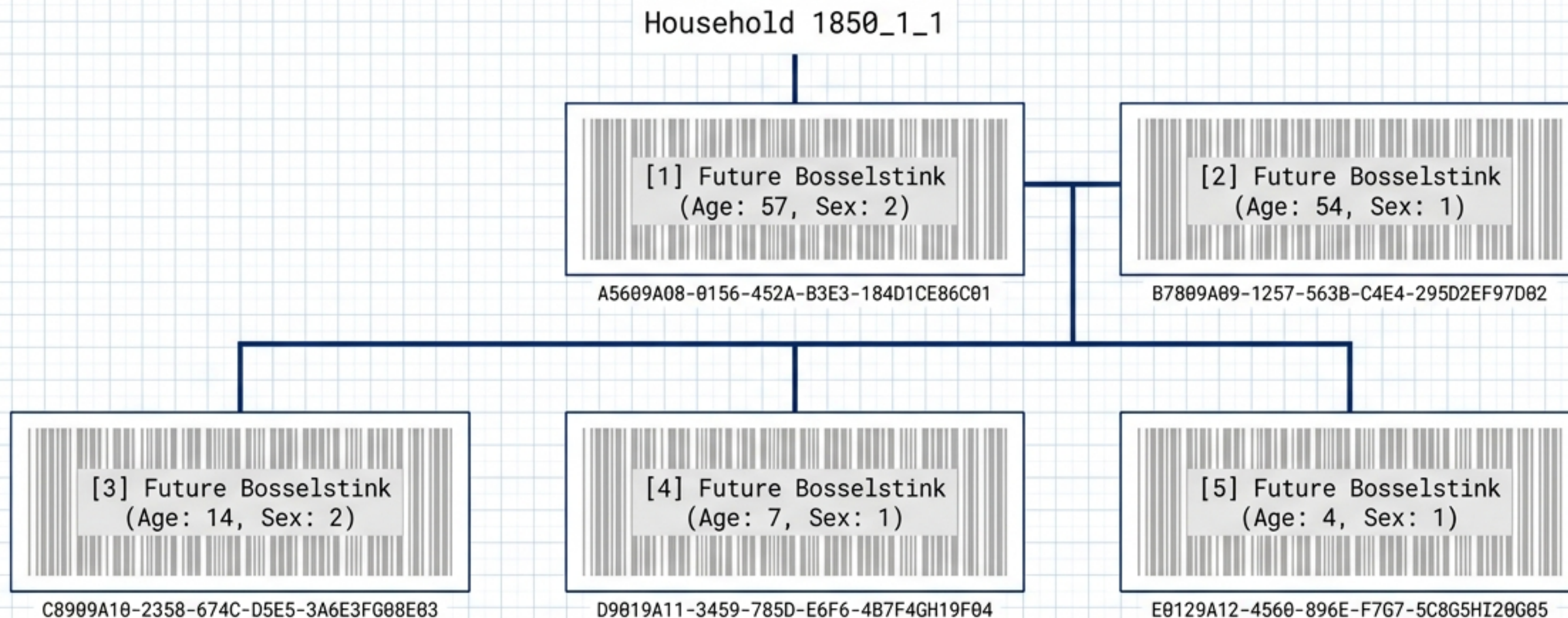
A Technical Walkthrough of the
Census-First Naming Architecture

System: GedcomNameOverlay.py
Environment: Python / DuckDB OLAP



Thomas Askey

A mathematically proven family tree without faces



Insight: 75% of raw historical census data lacks transcribed names. Our engine groups these people perfectly using relational data (POPLOC/MOMLOC), but assigns them the highly unique string **Future Bosselstink** as a placeholder. We have the skeleton, but we need the identity.

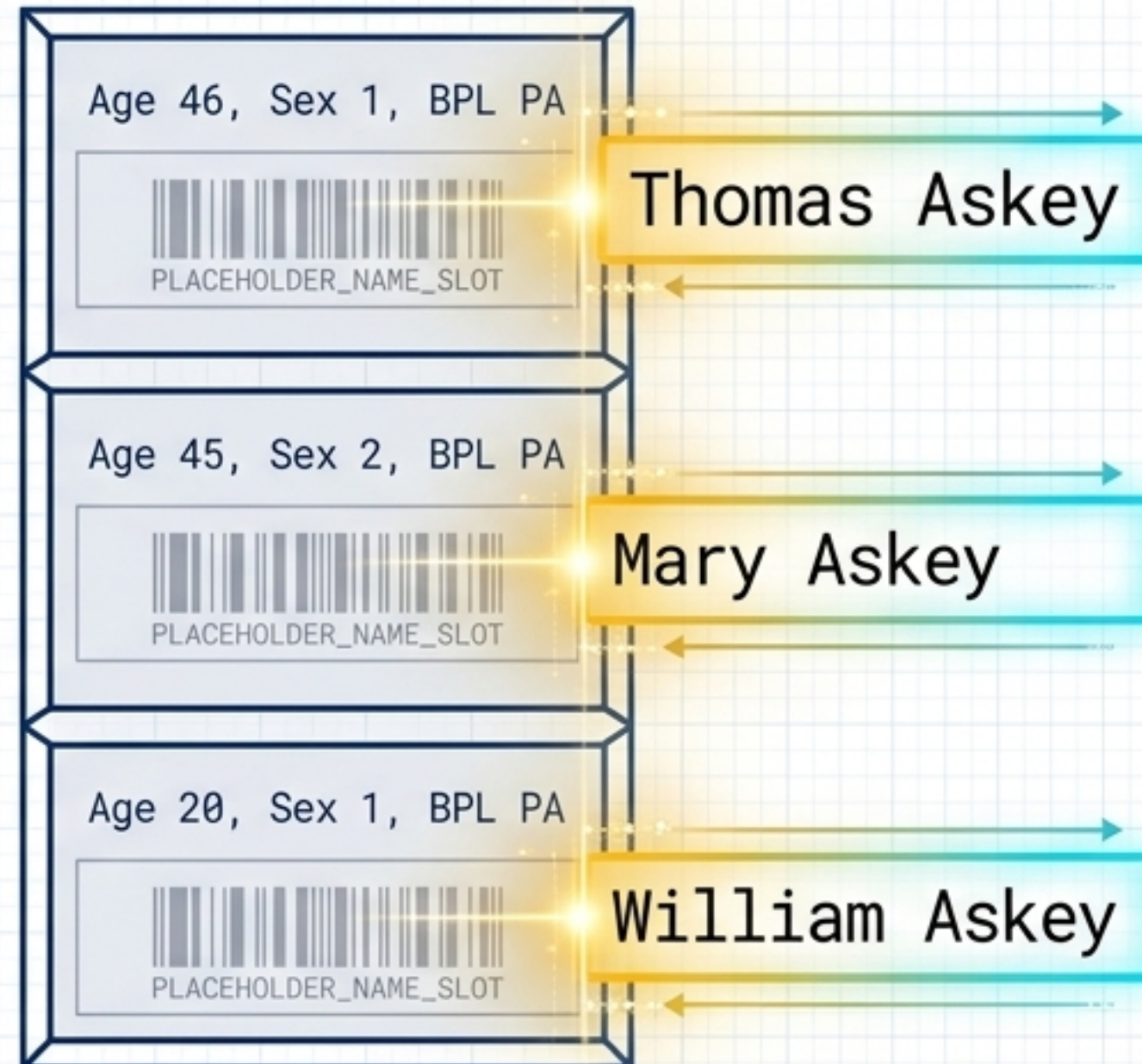
Snapping historical names onto a structural blueprint

The Vault

MasterVault_Relational1.db



The Overlay

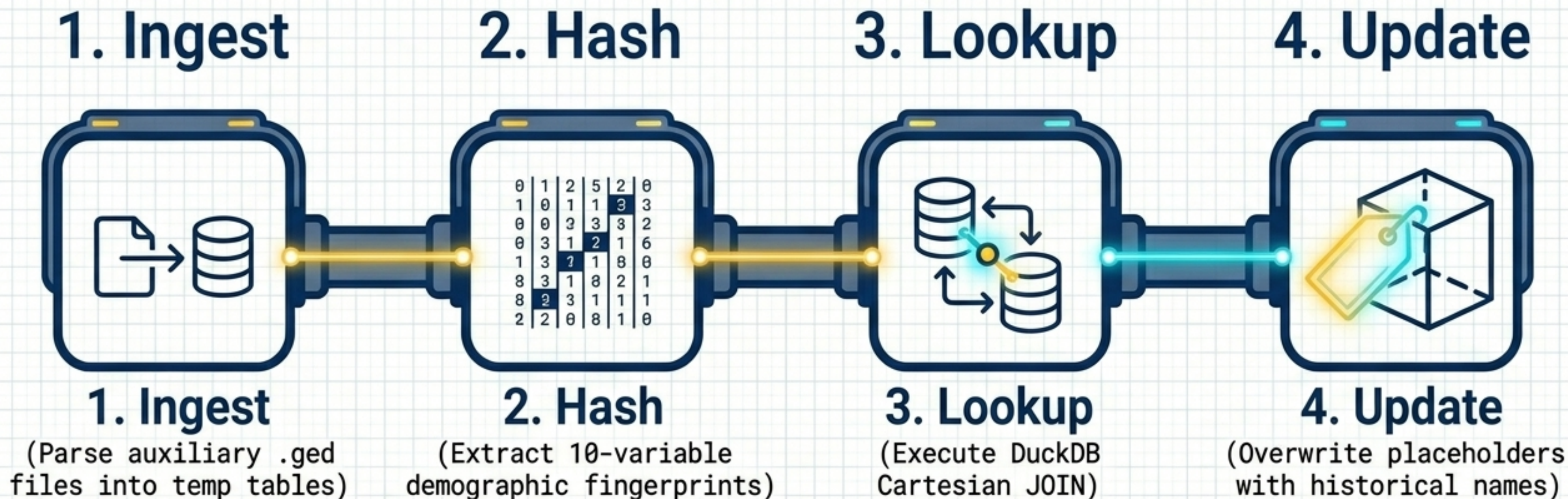


The GEDCOM

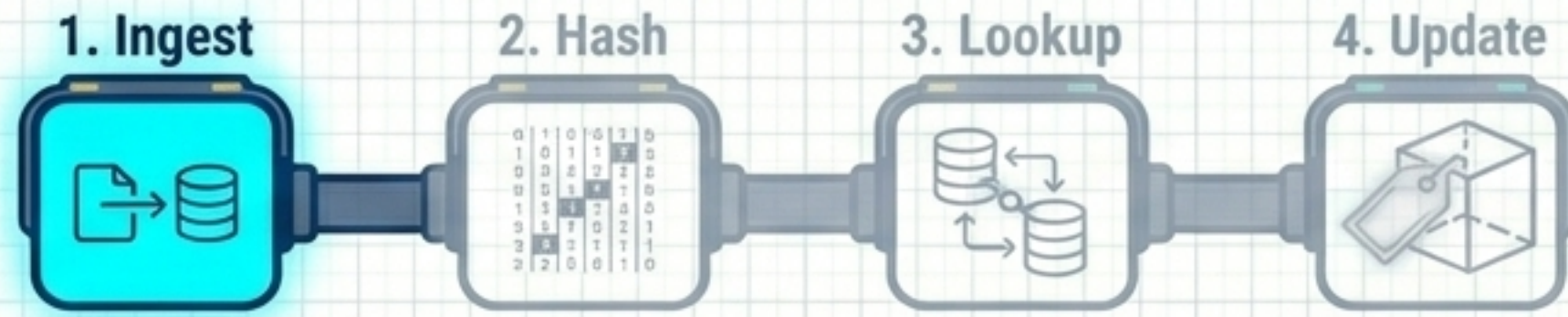
gedcom_sources/*.ged



The 4-Step Execution Pipeline

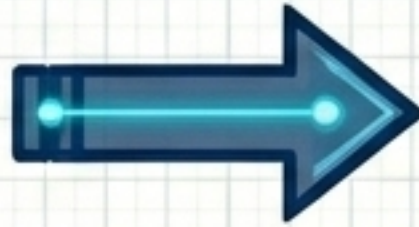


The `GedcomNameOverlay.py` script executes a linear pipeline. It translates both the raw census data and the external GEDCOM names into a shared mathematical language to map them together.



Step 1: Ingesting the auxiliary GEDCOM files

```
0 @I1@ INDI
1 NAME Thomas /Askey/
1 BIRT
2 DATE 1840
```



DuckDB Workspace		
ID	NAME	BIRT_DATE
@I1@	Thomas Askey	1840

Source Path: GEDCOM_INPUT_DIR = 'gedcom_sources'

Action: Scans for .ged files (Personal, WikiTree) and parses hierarchical text into a flat, queryable memory structure.

Step 3: Executing a memory-intensive OLAP lookup

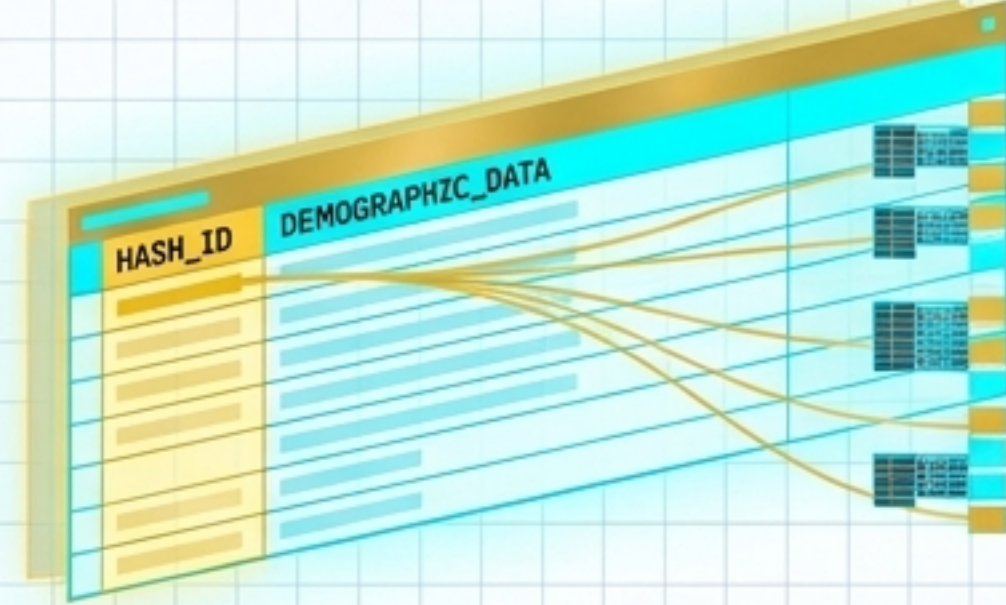
1. Ingest

2. Hash

3. Lookup

4. Update

TEMPORARY GEDCOM HASH TABLE

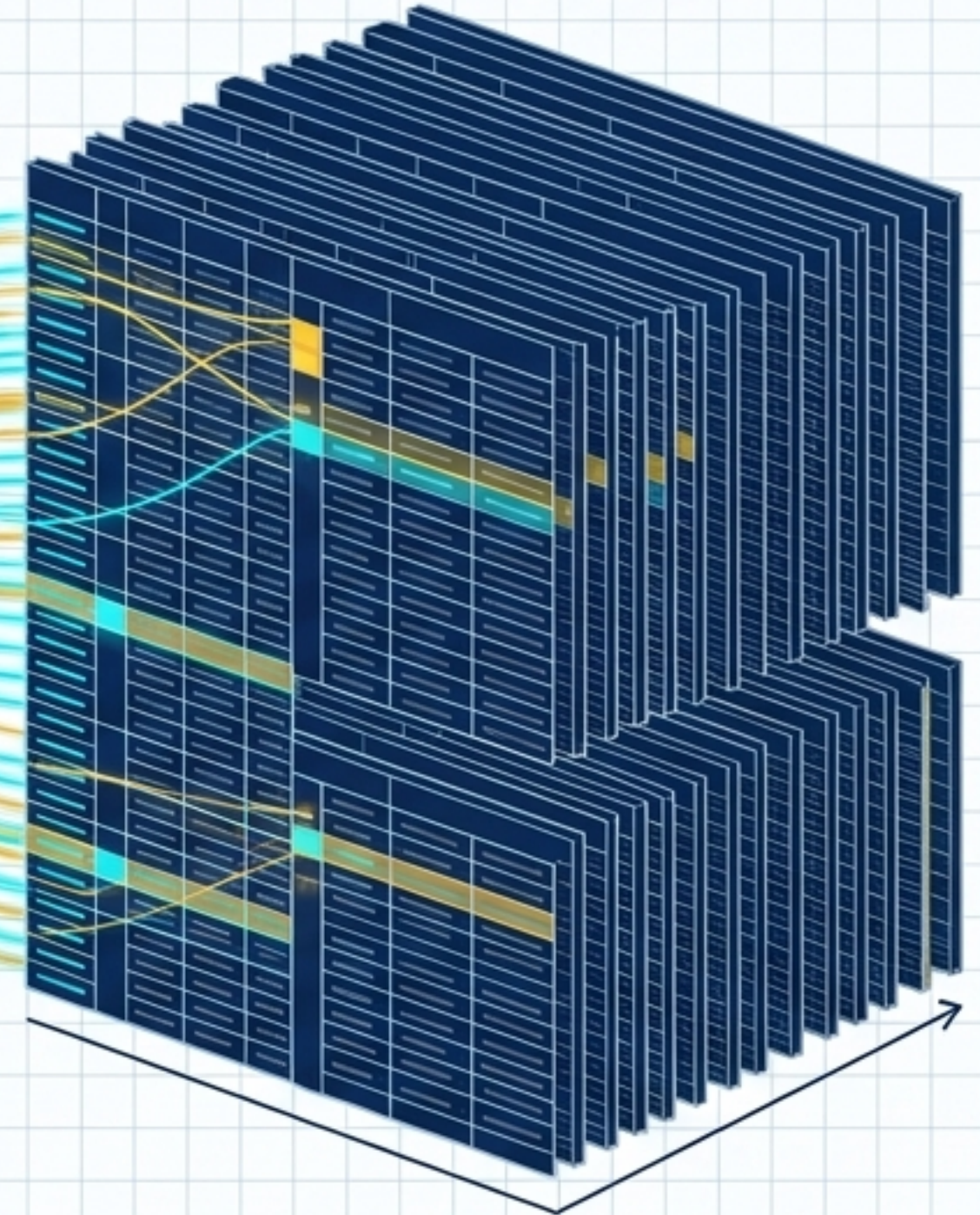


Engine: DuckDB (In-process OLAP, bypassing SQLite row-by-row limits).

Memory: PRAGMA memory_limit='90GB' allocated for massive parallel execution.



MasterVault_Relational.db



Strict Uniqueness Constraint: `HAVING COUNT(*) = 1.`

If a demographic hash isn't 100% unique nationwide, it is automatically discarded to protect data integrity.

1. Ingest

2. Hash

3. Lookup

4. Update

Step 4: Painting names over the nameless placeholders

Future Bosse1stink

Thomas Askey

UPDATE Logic Mechanics: When a unique 1-to-1 match is found via the demographic hash, the script targets the 'individuals' table and physically overwrites the first and last name placeholder variables with the true historical identities.

Preserving historical truth through database immutability



READ-ONLY / IMMUTABLE

MasterVault_Relational.db

UUID	Name
A5609A08...	Future Bosselstink
A5609A08...	Future Bosselstink
A5609A08...	Future Bosselstink
A5609A0F...	Future Bosselstink
A5609A0F...	Future Bosselstink

NEW WRITE

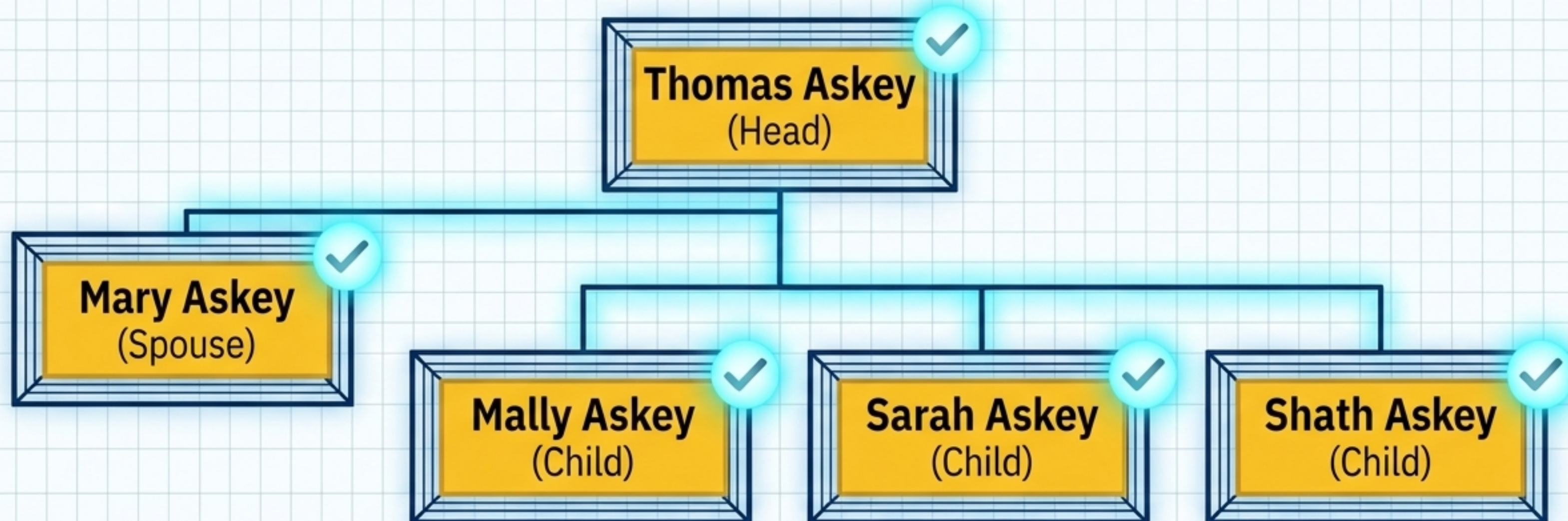


MasterVault_Named.db

UUID	Name
A5609A08...	Thomas Askey
A5609A08...	Mary Askey
A5609A08...	Mally Askey
A5609A0F...	Sarah Askey
A5609A0F...	Shath Askey

`GedcomNameOverlay.py` never alters the raw ground-truth vault. It explicitly creates a brand-new database to store the name changes. Downstream processes perform a **JOIN** between the two, keeping the raw census muscle permanently untouched.

The final output: A completely named, verifiable Golden Record



The **Label-Maker** script successfully bridges nameless mathematical structures with named historical records. The result is an export-ready, completely sourced family tree strictly adhering to GEDCOM 7.0 standards.